

إزاي تشرح الرسالة — دليل الكلام

العرض: الجمعة 11 سبتمبر 2026 · 29 سلايد · حوالي 25 دقيقة

الملف ده مش نص الرسالة. ده اللي بنقوله بصوتك.

اتعمل مخصص على أساس التلات حاجات اللي حصلت المرة اللي فاتت:

اللي حصل	الحل هنا
اتسألت في الكود ومعرفتش تجاوب	القسم 4 — جملة واحدة محضرة لكل module
الداتا كانت صعبة تتقرا، وإنت كمان	القسم 3 — تفتح إيه بالطبط، وتقول إيه
اللجنة مكاتتش عارفة يعني إيه CI/CD ولا GitHub	القسم 1 — أهم دقيقتين في العرض كله

القسم 1 — الافتتاحية: اشرح CI/CD و GitHub (دقيقتين)

القاعدة الأساسية

ابدأ بالشرح ده من غير ما حد يسألك. ما تقولش "زي ما إنتوا عارفين"، وما تسألش "حضراتكم متابعين GitHub؟" — اللجنة هتقول أيوه علشان متتخرجش، وبعدين مش هتفهم أي رقم بعد كده.

اعتبرها سلايد طبيعية في العرض، مش اعتذار.

الكلام بالظبط (اتمرن عليه بصوت عالي ٣ مرات)

تخلوا معايا إن كل مرة طالب يسلم فصل من رسالته، الجامعة تشغل عليه أوتوماتيك برنامج يراجع التنسيق، ويكشف الاقتباس، ويتأكد إن المراجع مكتوبة صح — وبعدين بيعتله رد: مقبول، ولا مرفوض. مفيش إنسان بيعمل ده. مكنة بتعمله، في كل مرة.

GitHub هو الدولايب المشترك اللي الشغل كله متسجل فيه — مين غير إيه، وامتى.

commit هو التعديل الواحد المحفوظ. زي نسخة من المستند، ومعاها ملاحظة من صاحبها بيقول فيها غير إيه. **CI/CD** هي المكنة اللي بتفحص أوتوماتيك. في البرمجيات، هي بتبني البرنامج كله من الأول وتشغل عليه آلاف الاختبارات، في كل مرة أي مبرمج يحفظ تعديل. لو كل حاجة عدت، التعديل سليم. لو حاجة فشلت، لازم واحد يوقف ويصلح.

المشكلة إن الفحص ده بطيء وغالي. بياخد من دقائق لساعات، ويكلف فلوس حقيقية على السيرفرات. ولما يفشل، المبرمج غالباً بيكون مشي على حاجة تانية ولازم يرجع.

وبحني ببسأل: في اللحظة اللي التعديل بيتحفظ فيها، قبل ما المكنة تشتغل خالص، نقدر نتوقع إنها هتفشل؟

بعدها قول التلات جمل دي على طول

1. «جمعت ٩٧٧٢ فحص حقيقي من ١٨ مشروع مفتوح المصدر معروفين — React, PyTorch, Rust, Python نفسها — من ال API الرسمي بتاع GitHub.»
2. «١١٪ بس منهم اللي بيفشل. ودي اللي بتخلي المسألة صعبة: لو أنا بس خمّنت "هينجح" في كل مرة، أكون صح ٨٩٪ من الوقت، وأكون متعلمتش أي حاجة. علشان كده ال accuracy مقياس مالوش لازمة هنا، وأنا بقيس بحاجة اسمها F1 بتكافئني بس لما أمسك الفشل.»
3. «والشرط الصعب اللي فرضته على نفسي: إني أستخدم بس المعلومات الموجودة قبل ما المكنة تشتغل. وده اللي يخلي التوقع مفيد — لأن توقع بعد ما الفلوس تتصرف مبيوقرش حاجة لحد.»

سلايد ٤ — الرسمة: تمشي عليها إزاي

سلايد ٤ رسمة بتحكي نفس الكلام بالصورة. متقراهاش — أشّر عليها بإيدك وإن بتتكلم، من الشمال لليمين:

«بصّوا معايا على الرسمة دي، هي الموضوع كله.

المربع الأول ده المبرمج وهو بيحفظ تعديل واحد. ودي كل المعلومات اللي بقى عندي في اللحظة دي: كام ملف اتغيّر، كام سطر، الكلام اللي كتبه، الفرع، والساعة.

المربع الكبير في الوسط ده المكنة وهي شغالة. أربع خطوات — بتجيب الكود، بتنزل الأدوات، بتبني البرنامج من الأول، وبعدين بتشغل كل الاختبارات. وشوفوا الأرقام اللي تحت كل خطوة: **من دقيقة لتسعين دقيقة**. ومحدث بيتدخّل في ده خالص.

وفي الآخر فيه احتمالين بس: نجح، وده ٨٩٪، أو فشل، وده ١١٪.

والمربع الأصفر ده أهم حاجة في السلايد: **أنا بتوقع هنا، عند المربع الأول**. كل اللي على اليمين ده لسه محصلش. ولو استنيت لحد ما المكنة تخلّص وبعدين قلت "دي كانت هتفشل"، مكنتش وقرت على حد ولا دقيقة، لأن الفلوس بتكون اتصرفت خلاص.»

نقطة تحضير: لو ممتحن سأل «طيب وليه مستخدمتش مدة البناء، هي أقوى إشارة؟» — الجواب موجود في الصندوق اللي تحت في نفس السلايد. أسّر عليه وقول: «مدة البناء **نتيجة** للفشل، مش سبب له. لو حطّيتها كان رقمي طلع أعلى، والنظام مبقى مالوش لازمة.»

لو حد قاطعك وسأل "يعني إيه GitHub؟"

«هو المكان القياسي اللي البرمجيات في العالم كلها متخزنة فيه والتعديلات عليها متسجّلة. حوالي ١٠٠ مليون مبرمج بيستخدموه. اعتبروه أرشيف دقيق جداً بيحفظ تاريخ كامل لمين غيّر إيه.»

كلمات تتجنبها في الشرح الأول

ما تقولش	قول بدالها
repository	المشروع / الكود
commit	تعديل محفوظ
workflow run / build	فحص أوتوماتيك واحد
pipeline	عملية الفحص الأتوماتيكي
pre-execution features	معلومات متاحة قبل ما الفحص يبدأ
class imbalance	١١٪ بس من الحالات فشل
the positive class	الحاجة اللي يحاول أكتشفها: الفشل

القسم 2 — ليه المشكلة دي مهمة (سلايد 5 و 6)

قول الأرقام دي بصوت واثق، وما تستعجلش عليها:

- ٨٩٪ بينجح ١١٠٪ بيفشل
- ٣.٤٥ workflow run لكل commit — «ال commit الواحد بيشتغل أكثر من workflow run، والنقطة دي هتفرق معايا جداً بعد شوية»
- الفشل بيكلف compute + ١٥ دقيقة من تركيز المبرمج

الجملة اللي تقفل بيها:

«فالسؤال مش أكاديمي. لو عندي ألف workflow run في اليوم، يبقى فيه ١١٠ فشل كل يوم، بيصرفوا موارد ووقت مش بيرجعوا.»

القسم 3 — الداتا: تعرضها إزاي (سلايد ٩ و ١٠)

القاعدة

ما تفتحش ال CSV الخام قدامهم. ٩٧٧٢ صف في ٢٠ عمود — مقروء مستحيل، ودي بالظبط اللي حصلت المرة اللي فاتت.

افتح Dataset_Guide.xlsx . فيه ٤ صفحات:

الصفحة	تقول عليها إيه
Summary	«الملخص: منين جبت الداتا، وكام صف، والتوزيع»
Data Dictionary	«كل عمود ومعناه بالكلام العادي، والأخضر معناه بستخدمه، والأحمر معناه استبعدته ومكتوب السبب»
Sample Rows	«عشرين صف حقيقي — عشرة نجحوا وعشرة فشلوا — علشان تشوفوا شكل الداتا»
By Repository	«failure rate لكل مشروع — من صفر٪ لـ ٣٨.٣٪»

الأربع حاجات اللي تقولها قبل ما حد يسألك

1. «استبعدت run_attempt و run_duration_sec .»

«الأتنين دول معروفين بس بعد ما ال workflow run يخلص. مدة ال workflow run هي نتيجة الفشل مش سببه. لو استخدمتهم كان الرقم بتاعي هيطلع أحلى، والنظام هيقى مالوش قيمة، لأنه مش هيقدر يتوقع غير الماضي.»

2. «عمود files_changed متقطوع عند ٣٠٠.»

«ال API بتاع GitHub بيرجع أول ٣٠٠ ملف بس. فعندي ١١٠ صف مكتوب فيهم ٣٠٠ بالظبط وممكن يكونوا أكثر. أنا بقول ده بنفسي، مش مخبي.»

«ده غلط في التجميع — قرينه من endpoint مبيرجّعوش. العمود اتشال، ويقوله علشان الشفافية.»

4. «الصفوف workflow runs، بس ال features بتوصف ال commits.»

«ودي أهم حقيقة في الداتا كلها، ومنها جاء الغلط اللي صلّحته — هرجع لها في سلايد ٢٠.»

سلايد ١٠ — الفرق بين المشاريع

بعد سلايد الداتا فيه سلايد رسمة بتوري failure rate لكل مشروع. دي الرسمة اللي تشرحها بالراحة، لأنها هي السبب في أهم نتيجة عندك:

«كل سطر هنا مشروع، والعمود اليمين ده عدد ال workflow runs اللي فشلت فيه.
بصّوا على السطر اللي تحت خالص: elastic/elasticsearch — ٦٠٠ workflow run، وولا فشل واحد. وبصّوا على السطر اللي فوق: prisma — ٢٣٠ فشل من ٦٠٠.
يعني failure rate ماشية من صفر٪ لـ ٣٨٪ — و١١ ضعف بين السبعناشر مشروع اللي بيفشلوا أصلاً. ودي الحقيقة اللي بتفسّر النتيجة اللي هتشوفوها بعد شوية: إن اسم المشروع لوحده بيتوقّع أحسن من الموديل الكامل.»

وقول التحفّظ بنفسك على طول:

«وده كمان بيحدّد النظام: مشروع جديد ملوش تاريخ، فالموديل ملوش حاجة يستند عليها. علشان كده أنا مش بقول إن النتائج دي بتنفع على أي مشروع بره ال ١٨ دول.»

⚠ السؤال اللي هيتسأل حتماً: «إزاي elasticsearch صفر؟»

الرقم ده أكثر حاجة في السلايد هتشد نظر ممتحن. الإجابة محضّرة، والأرقام من الداتا الخام:

الحقيقة	الرقم
شغلانات checksum و documentation	٥٩١ من ٦٠٠ = ٩٨٪
عدد ال workflows اللي عنده على GitHub Actions	٦ بس (nest عنده ١٣٧ ، tensorflow ١٠٧)
النافذة الزمنية لل workflow run ٦٠٠	١٣٨ ساعة

الكلام:

«سؤال مهم وأنا فحصته. النسبة صفر مش لأن Elasticsearch مشروع مثالي — لأنه مبيشغلش ال CI الحقيقي بتاعه على GitHub Actions أصلاً، بيشغله على Buildkite.

اللي جمعته منه ٦٠٠ workflow run، ٥٩١ منهم — ٩٨٪ — نشر documentation والتأكد من checksum بتاع ملف. مفيش فيهم بناء ولا tests. حاجات زي دي بنويهاً مبيتفشلش.

وده مش بيضعف النتيجة، بيفسرها: workflow_name واحد من الأربع features اللي ديتها للموديل، فهو بيتعلم إن docs-deploy مبيتفشلش. ودي معلومة حقيقية ومتاحة عند التوقع — بس هي أبسط بكثير من إني أتوقع إن commit معين هيكسر ال build.

والقراءة الأمينة: ال workflow run ٩٧٧٢ بتاعتي مش كلهم بنفس الثقل. فيهم tests حقيقية وفيهم شغل إداري. ودي مكتوبة في الرسالة، حد رقم ٧ في قسم ٨.٣.»

وفي سلايد ٢٦ (Honest Limitations): السلايد فيها ٦ حدود مكتوبة، والحد ده التاسع مش عليها — قوله بصوتك: «وفيه حد سابع في الرسالة: ال workflow runs اللي جمعتها مش كلها بنفس الثقل — فيها tests حقيقية وفيها نشر documentation، Elasticsearch هو الحالة القصوى.» كده تكون قلته إنت، مش هما.

ملاحظة أداء: الجملة الأخيرة هي اللي بتقفل السؤال — إنك تقول «مكتوبة في الرسالة» بيحول النقطة من عيب اكتشفوه لحد إنت مسجله.

القسم 4 — الكود: تتكلم عنه إزاي (سلايد ١١ و ١٢)

القاعدة

المرة اللي فاتت السؤال في الكود وقعك. الحل مش إنك تحفظ الكود — الحل إنك تحفظ جملة واحدة لكل ملف تقول هو مسؤول عن إيه.

سته ملفات بس هي المهمة — احفظ الجدول ده

الملف	«الملف ده مسؤول عن...»
collect_github_data.py	«الكلام مع الـ API بتاع GitHub. بيطلع ملف الداتا الخام. مشغلوش تاني بعد كده.»
data_preparation.py	«التنظيف، وتصنيع الـ features، وتنظيف النصوص، وكل عمليات تقسيم الداتا.»
hybrid_pipeline.py	«الـ transformer بأربع فروع، والتلات موديلات.»
cross_validation.py	«طريقة التقييم المجمعّة على الـ commit، واختيار الـ threshold.»
train_evaluate.py	«المقاييس، والـ ablation، وحساب القيمة المالية.»
run_corrected_evaluation.py	«المشغل الرئيسي. بيطلع كل رقم وكل رسمة في الفصل السابع.»

سلايد ١٢ — الـ ١٦ feature: تشرحها إزاي

السلايد أربع أعمدة. كل عمود نوع مدخل مختلف، وكل نوع محتاج معالجة مختلفة — ودي السبب في وجود الـ ColumnTransformer.

العمود	كام	يعني إيه بالعربي
NUMERICAL	٥	أرقام متصلة — حجم ال commit وطول الرسالة
CATEGORICAL	٤	أسماء — مين المشروع وأي workflow وأي فرع وإيه اللي شغّله
BINARY	٦	أسئلة إجابتها أيوه/لا — صفر أو واحد
TEXTUAL	١	نص رسالة ال commit

$$١٦ = ١ + ٦ + ٤ + ٥$$

ال features واحدة واحدة — احفظ العمود الأخير

NUMERICAL (٥):

feature ال	يعني
log_lines_added	عدد السطور المضافة، بعد $\log(1+x)$
log_lines_deleted	عدد السطور المحذوفة، بعد $\log(1+x)$
log_files_changed	عدد الملفات المتغيّرة، بعد $\log(1+x)$
commit_message_length	طول الرسالة بالحروف
avg_lines_per_file	متوسط السطور لكل ملف = $\text{total_changes} / (\text{files_changed} + 1)$

ليه $\log$ ؟ لأن التوزيع ملتوي جداً — معظم ال commits صغيرة وفيه شواذ بآلاف السطور. ال $\log$ بيضغط الشواذ علشان متسيطرش على ال StandardScaler. وال +1 في $\log(1+x)$ علشان $\log(0)$ مالهاش قيمة.

ليه 1.0 + في المقام؟ علشان مقسّمش على صفر لو files_changed صفر.

CATEGORICAL (٤): repository · workflow_name · branch · event

دي أهم أربع features في المشروع كله — هما اللي في تجربة categorical_only اللي كسبت الموديل الكامل.

BINARY (٦) — وال thresholds بالطبط:

feature	التعريف	= في
is_large_commit	total_changes < median (٣٥ سطر)	٤٩٩٤٪
is_many_files	files_changed < median (٢ ملف)	٤٨٨١٪
is_weekend_commit	يوم سبت أو حد	٩٩٪
is_off_hours_commit	الساعة بره ٦ ص - ٦ م (UTC)	٥١٧٪
is_bot_author	اسم الكاتب فيه نمط بوت	١١١٪
was_truncated	الرسالة ≤ ١٠٠٠ حرف (حد التجميع)	١٠٠٤٪

TEXTUAL (١): commit_message_clean — رسالة ال commit بعد التنظيف.

ال Treatment — السطر اللي تحت كل عمود

المعالجة	بتعمل إيه وليه
StandardScaler	بتخلّي كل رقم متوسطه صفر وانحرافه واحد. لازمة لأن Logistic Regression بتتأثر بمقياس الأرقام — من غيرها commit_message_length (مئات) بتغلب log_files_changed (١-٣)
OneHotEncoder	بتحوّل كل قيمة نصّية لعمود صفر/واحد. ١٨ مشروع = ١٨ عمود. و«rare-category bucketing» يعني القيم النادرة بتتجمّع في خانة other علشان الأعمدة متنفجرش
Passthrough	مبتعملش حاجة — ال features دي أصلاً صفر أو واحد، مفيش حاجة تتقاس
TF-IDF	بتحوّل النص لأرقام. كل كلمة بتاخذ وزن: بتتكرر كثير في الرسالة دي = وزن أعلى، بتتكرر في كل الرسائل = وزن أقل. ngram 1-2 يعني كلمات فردية وأزواج، max_features=3000 يعني أكثر ٣٠٠٠ كلمة أهمية بس

«ال TF-IDF كان يتعلّم مين كتب ال commit مش إيه ال commit. لقيت أسماء مؤلفين في أهم الكلمات المميّزة. فبنيت قائمة ٦٩٣ كلمة من أسماء المؤلفين وأسماء المشاريع وتوقعات البوتات، وال vectorizer بيتجاهلهم. لأن موديل بيتوقع الفشل من اسم المبرمج ده موديل مالوش قيمة.»

⚠ لو سألوك «إيه هي كل ال ١٦؟»

ما تحاولش تعدّهم من دماغك. قول:

«خمس أرقام لحجم ال commit، أربع تصنيفات لسياق المشروع، ست مؤشرات صفر/واحد للوقت وحجم ال commit ونوع الكاتب، ونص واحد هو رسالة ال commit. القائمة الكاملة بالأسماء في السلايد وفي قسم ٦ في الرسالة.»

كفاية كده. مش مطلوب تحفظ ١٦ اسم — مطلوب تعرف الأربع مجموعات وليه كل واحدة بتتعالج بطريقة مختلفة.

سؤال ال architecture — الأربع فروع

«عندي ColumnTransformer واحد يدمج أربع أنواع مدخلات في مصفوفة واحدة: الأرقام بيتعمل لها StandardScaler علشان توحيد المقياس؛ والتصنيفات — المشروع وال workflow والفرع والحدث — بيتعمل لها OneHotEncoder علشان تتحول لأعمدة صفر وواحد؛ وال flags اللي أصلاً صفر وواحد بتمر زي ما هي؛ ونص رسالة ال commit بيتعمل له TF-IDF علشان الكلمات تتحول لأرقام حسب تمييزها.»

«ال labels بتاعتي كلمات: success و failure، مش صفر وواحد. موديلين من التلاتة بيقبلوا الكلام. XGBoost بيطلب أرقام. فبدل إني أحول ال labels في المشروع كله — وده كان هيسهل إني أتوه وأنسى مين رقمه واحد — لقيت XGBoost في طبقة رقيقة بتحول عند الدخول وترجع عند الخروج. فالتلات موديلات بقى لهم نفس الواجهة بالطبط، وكود التقييم مش ممكن يعاملهم بطريقة مختلفة بالغلط.»

لو سألوك سؤال في الكود مش عارف إجابته

ما تخترعش. قول:

«مش فاكّر التفاصيل دي بالطبط دلوقتي، ومش عايز أقول رقم غلط. الملف موجود ومعايا والكود كله في الحزمة، وأقدر أوريكم السطر بالطبط. الفكرة العامة إن...»

القسم 5 — النتائج: تعرضها إزاي

الرقم الأساسي

$F1 = 0.4216 \pm 0.0638$

أول حاجة تقولها بعد الجدول (ما تستناش حد يلاقيها)

«التلات موديلات دول مش مختلفين إحصائياً. الفرق بينهم أقل من ١٪، والانحراف المعياري بين ال folds أكبر من ٦٪. فأنا مش هقول إن فيه موديل "فايز" على أساس ال F1 — ده ادّعاء مش قادر أسنده. اخترت XGBoost على أساس ال PR-AUC، وهو المقياس المناسب مع عدم التوازن.»

النتيجة الأهم عندك (سلايد ١٦)

لو الوقت ضيق، دي اللي تقولها:

«عملت تجربة اسمها categorical-only: إديت الموديل بس اسم المشروع، واسم ال workflow، والفرع، والحدث اللي شغلته. ومقولتلوش أي حاجة عن حجم ال commit ولا عن اللي المبرمج كتبه. والنتيجة إن التجربة دي كسبت الموديل الكامل بتاعي في كل مقياس — 0.4808 مقابل 0.4216. ودي بتكدّب الفرضية اللي بدأت بيها المشروع. والقراءة الآمنة إن النظام بتاعي في الأساس مُقدّر خطورة على مستوى المشروع، مش على مستوى ال commit الواحد. هو بيتوقع إن ال workflow run في مشروع تاريخه سيء غالباً هيفشل — وده مفيد، بس هو ادّعاء أضعف من إني أتوقع إن commit معين هو اللي هيكسر ال build. وأنا اللي عملت التجربة دي بنفسني، لأنها أخرج سؤال ممكن ممتحن يسأله.»

السبب: failure rate ماشية من صفر٪ لـ ٣٨.٣٪ بين المشاريع (١١ ضعف بين اللي بيفشلوا أصلاً) — من صفر٪ في Elasticsearch (٦٠ workflow run وولا فشل واحد) لـ ٣٨.٣٪ في Prisma.

القسم 6 — أصعب موقف: إن الأرقام نزلت (سلايد ٢٠ و ٢١)

ده أهم سؤال ممكن يتسأل. والإجابة دي بتقلب الموقف لصالحك

اتمرن على الكلام ده أكثر من أي حاجة تانية في الملف.

«أيوه، والأرقام نزلت، وعايز أشرح ليه بالطبط، لأن ده أكثر نتيجة أنا واثق منها.

أنا كنت بقول قبل كده F1 يساوي 0.5924. دلوقتي بقول 0.4216 — لنفس الموديل على نفس الداتا. الموديل مبقاش أسوأ. المسطرة اللي بقيس بيها هي اللي بقت أمينة.

لقيت غلطتين في طريقة التقييم.

الأولى: الداتا بتاعتي فيها ٩٧٧٢ workflow run، بس من ٢٨٣٥ commit بس. يعني نفس ال commit بيظهر كذا مرة. وال split القديم بتاعي كان بيوزع الصفوف عشوائي، فنسخ شبه متطابقة من نفس ال commit كانت بتقع في التدريب والاختبار مع بعض. فالموديل كان بياخد درجات على إنه "عرف" حاجة كان شافها قبل كده.

الثانية: أنا كنت بختار ال threshold باني أجرب كل القيم وأشوف أنهي واحدة بتطلع أعلى F1 على ال test set، وبعدين أقول الرقم ده هو نتيجتي على نفس ال test set. ودي حلقة مفرغة.

صلّحت الاثنين. ولأن كل تصحيح ممكن يتطبّق لوحده، أنا قادر أقول التأثير بالطبط: ٠.٠٦٠ من الغلطة الأولى، و٠.١٢٦ من الثانية.

والحاجة اللي مكنتش متوقعها: غلطة ال threshold — اللي هي الأقل وضوحاً — كلّفت أكثر من ضعف غلطة ال leakage.

وأنا اللي لقيت ده بنفسني، وقّسته، وصلّحته، ونشرت التصحيح جوه الرسالة نفسها في قسم ٧.٤.٥.

سلايد ٢١ — الرسمة اللي بتقول نفس الكلام

بعد سلايد الأرقام فيه رسمة أعمدة. دي أحسن حاجة تأشّر عليها وانت بتجاوب:

«العمود الأول ده الرقم القديم. العمودين البرتقاني اللي بينزلوا دول الغلطتين، كل واحد لوحده — والعمود الأخير هو الرقم الأمين.

والنقطة إن كل خطوة اتعملت لوحدها، فأنا قايس تكلفة كل غلطة، مش مقدّرها. وشوفوا إن العمود الثاني — غلطة ال threshold — أطول من الثالث أكثر من الضعف.»

لو حد قال "يعني كنت غلطان قبل كده؟"

«أيوه، في طريقة القياس. ودي الحاجة اللي المنهج العلمي المفروض يعملها. Kapoor و Narayanan لقوا نفس نوع الغلطة ده في ٢٩٤ ورقة بحثية في ١٧ مجال مختلف — يعني هي غلطة معروفة وتنفوت على ناس كثير، لأن كل خطوة في التحليل بتبان صح لوحدها. الفرق إني لقيتها وقستها بنفسي.»

القسم 7 — سؤال «ليه رقمك أقل من الأبحاث التانية؟»

فيه ورقة جديدة (٢٠٢٥) بتقول $F1 = 0.88$. لو حد ذكرها:

«صح، وأنا بستشهد بيها في الرسالة كمرجع ٢٧. بس الرقمين مش قابلين للمقارنة، لتلات أسباب أنا قستهم من جداول الورقة نفسها:

واحد: أقوى feature عندهم — بال SHAP — هي مدة الـ build. ودي مش معروفة غير بعد ما الـ build يخلص. فرقمهم بيقول إيه اللي ممكن تحققه لما تسمح لنفسك بمعلومات ما بعد التنفيذ، وأنا رافض ده أصلاً.

اتنين: failure rate عندهم ٢٢٪ — ١١٢,٨٦٠ من ٥١٣,٠٠٠. عندي ١١٪. وكل ما الفئة الأقلية تقل، المسألة تصعب، والـ $F1$ نفسه حساس للنسبة دي. فالرقمين مش نفس الكمية.

تلاتة: المتغير بتاعهم أربع فئات مش اتنين، وهما نفسهم بيقولوا إن الأداء على الفئات الصغيرة أضعف. فإحنا مش بنقيس نفس الحاجة.

والدليل من ناحية تانية: الـ accuracy بتاعتهم ٨٩.٧٪ — وتوقع ثابت بـ "هينجح" بياخد ٦٦.٨٪ على الداتا بتاعتهم و٨٩.٠٣٪ على بتاعتي.»

القسم 8 — القيمة المالية (سلايد ٢٣)

قول الرقم، وبعده على طول قول التحفظ — التحفظ هو أقوى نقطة عندك:

«التقدير ٢٤٣,٦٧٠ دولار في السنة، صافي بعد خصم تكلفة الإنذارات الكاذبة.

بس أنا مش بعتمد على الرقم ده. لأنه متعلّق بافتراض أنا **مقستهوش**: إن الإنذار الكاذب بيكلّف ٢.٥ دولار. فبدل إني أدافع عن التقدير، أنا بقول **نقطة التعادل**: تكلفة الإنذار الكاذب اللي عندها كل إعداد يبطلّ يعوّض نفسه.

للموديل اللي اخترته: ٢٠.٤١ دولار. أما Logistic Regression اللي رقمه أحلى عند ٢.٥ دولار، نقطة التعادل بتاعته ١٠.٢٧ دولار بس — يعني حوالي تمن دقائق من انتباه مبرمج.

يعني الإعداد اللي تقديره الأعلى هو نفسه الأقل تحمّلاً لو الافتراض غلط. ودي قاعدة أفضل للقرار من رقم واحد.»

القسم 9 — قواعد الأداء

اعمل

- اشرح CI/CD في الأول من غير ما حد يطلب.
- قول الرقم الأمين، وبعدين اشرح ليه هو أمين.
- اذكر نواقصك قبل اللجنة. كل نقص في الملف ده إنت **قِسته** — وده قوة لو اتعرض صح.
- لو مش عارف: «مش عارف، بس اللي كنت هعمله علشان أعرف هو...»
- اتكلم بالراحة. اللجنة مش فاهمة المجال، فالسرعة عدوك.

ما تعملش

- ما تقولش إنك أحسن من الأبحاث السابقة. رقمك أقل، والسبب مشروع — قول السبب.
- ما تعتذرش عن 0.42. دي نتيجة مقيسة، مش نتيجة وحشة.
- ما تقولش «الموديل فشل» عن فرضية الـ hybrid. قول «التجربة كدّبت فرضيتي، وأنا بقول كده.»
- ما تخترعش رقم. لو نسيت: «الرقم في الفصل السابع، ومش عايز أقوله غلط.»
- ما تفتحش الـ CSV الخام.

الجملة اللي بتكسب الموقف لو ال leakage اتفتح

«أنا اللي لقيتها بنفسني، وقست بالطبط تأثيرها، وصلحتها، ونشرت التصحيح جوه الرسالة. وده اللي المنهج المفروض يعمل.»

القسم 10 — خطة الوقت

الجزء	سلايد	دقائق
مقدمة + شرح CI/CD + الرسمه	٤-١	٤٥
المشكلة والفجوة والأهداف	٨-٥	٣
الداتا + الفرق بين المشاريع	١٠-٩	٢٥
المعمارية وال features	١٢-١١	٢
طريقة التقييم	١٣	١٥
النتائج وال ablation	١٧-١٤	٣
ال threshold	١٩-١٨	١٥
الأرقام نزلت ليه + الرسمه	٢١-٢٠	٢٥
النتائج النهائية والقيمة	٢٣-٢٢	٢
الأهداف والمساهمات والنواقص	٢٦-٢٤	٢
المستقبل والخاتمة	٢٩-٢٧	١

الإجمالي ≈ ٢٥٥ دقيقة. لو عندك ٢٠ دقيقة بالطبط، اقصر في ٨-٦ و ٢٨-٢٧. ما تقصرش في ٤-١ ولا في ٢١-٢٠.

آخر حاجة — قبل ما تدخل

اتمرن على حاجتين بس بصوت عالي، ثلاث مرات لكل واحدة:

1. شرح CI/CD (القسم ١)

2. إجابة "ليه الأرقام نزلت" (القسم ٦)

الأتنين دول بيغطّوا أكبر خطر عندك. الباقي إنت عارفه.

ولو نسيت كل حاجة، افتكر رقم واحد: 0.4216.